

A Mutation Triggering Method for Genetic Algorithm to Solve Traveling Salesman Problem

Khandker M Qaiduzzaman¹, Maliha Afsa¹, Sadman Sobhan², Md. Elias Hossain¹,
Sabira Khatun^{3*}, Syamimi Mardiah Shaharum³

¹Daffodil International University, Dhaka 1207, Bangladesh

²Dohatec New Media, Dhaka 1000, Bangladesh

³Faculty of Electrical and Electronics Engineering,
Universiti Malaysia Pahang, Malaysia

Email: *sabirakhatun@ump.edu.my
nafees.kull@gmail.com

Abstract. This paper reveals the effect of combining mutation operators in Genetic Algorithm. Though mutation is a very effective genetic operator to escape the local optima, it has an adverse effect on computation time. It is quite a challenge to say how much probability is appropriate for mutation. The major contribution of this paper is to design a mutation triggering method by combining three mutation operators (Swap, Insertion and 2-Opt) and applying adaptive probability. To decide which mutation will be activated at a given generation, a decision-making method named Mutation Triggering Method is proposed. To measure the performance of the proposed method, the classical traveling salesman problem was considered. The experimental result shows that the proposed method was able to find a better solution (travel cost) than the other approaches. For computation time, the proposed strategy did as well as other mutation strategies. The paper concludes that the combination of several mutation operators can ensure the benefits of diversity as well as the benefits of faster convergence.

Keywords: Triggering, Adaptive, Mutation, Genetic, Hybrid, Operator, 2-opt, Swap, Insertion, TSP

1 Introduction

The benefit of solving traveling salesman problem (TSP) is that it has many real-world applications. The notable applications of classical TSP are vehicle routing, order picking problem in warehouses, computer wiring, drilling of the printed circuit board (PCB), X-ray crystallography, overhauling gas turbine engine, mask plotting in PCB production [1]. In this paper, the Genetic Algorithm (GA) is implemented for solving TSP. The framework of GA consists of genetic operators that could be modified easily according to the nature of the problem. The best thing is that if the operators of GA can be designed perfectly, it provides the optimal/near-optimal solution. It is inspired by the biological evaluation where each of the solutions is considered as a chromosome. Every chromosome is consisting of a set of biological elements, gene. The basic idea of GA is to generate some chromosome (i.e. solutions) randomly or based on some ideas (heuristic). These chromosomes are evaluated using a fitness function. The fitness function assigns a fitness value to each chromosome. After this phase, a selection is

performed to decide which chromosomes are fittest to survive on the next generation. The literature shows that there are many strategies of selection such as Roulette Wheel Selection, Rank Selection, Tournament Selection, etc. [2]. After the selection, the fittest chromosomes are called as parents and they get the chance of biological reproduction through the crossover. Crossover is one of the most important genetic operators that helps to converge to the solution. The basic idea of a crossover is to exchange genetic elements (i.e. gene) between two parents to generate new offspring. Numerous researches have been done on crossover operators and some popular crossover strategies are Partially Mapped Crossover (PMX), Cycle Crossover (CX), Order Crossover (OX), etc. [3]. Crossover is applied with a higher probability to converge faster. However, using only the crossover might converge the algorithm to local optima. For this reason, another genetic operator named mutation is introduced to GA. The mutation is liable for bringing diversity to the solution by tweaking some genetic materials of a few chromosomes. Some well-known mutation operators are Exchange or Swap Mutation (EM), Insertion Mutation (IM), Simple Inversion or 2-opt Mutation (SIM), etc. [3]. The mutation is applied with a very low probability P_m . Higher mutation probability results in divergence of the algorithm. Still, the mutation is considered the most crucial genetic operator to find a globally optimal solution. After the mutation, the old population are replaced by the new population (generated by crossover and mutation). That is how one generation of GA is completed. GA should be run for several generations to find an optimal solution.

In this work, we solved TSP which is a well-known NP-hard problem [4]. It consists of a set of cities and a salesman. The salesman has to visit each one of the cities starting from a certain one (also known as the hometown) and returning to the same city. The challenge of this problem is that the traveling salesman wants to minimize the total length of the trip [5]. Given, n cities and the objective of TSP is to find an integer permutation $\pi = c_1, c_2, c_3, \dots, c_n$ with c_n being the city n , that minimizes the sum, $(\pi) = \sum_{i=1}^{n-1} (c_i, c_{i+1}) + (c_n, c_1)$, where $d(c_i, c_{i+1})$ denotes the distance between city i and city $i+1$. When GA is implemented to solve TSP, each city is considered as a gene and each tour is considered as a chromosome.

TSP is represented as a graph and it is considered difficult to solve. For example, if the number of cities of a TSP problem is 10, there are $(n-1)! = (10-1)! = 3,62,880$ possibilities to try which is not wise to solve using Brute Force approach. Moreover, the complexity of the problem increases when the number of cities increases. The main challenge of this problem is to find an optimal solution within a short time. That is why we choose GA which is very effective because of its meta-heuristic nature, yet a further improvement of GA is needed. Especially, the genetic operators like selection, crossover and mutation play a vital role in the performance of GA. Good operator design can converge the algorithm quickly whereas bad operator design can take the same algorithm away from the optimal solution. We expect that the combination of genetic operators might give a better solution.

The literature shows that extensive studies have been done to solve TSP using GA in the last decades. In [6], a multi-objective solid traveling salesman problem is solved using R-MOGA. In this paper, a generation dependent mutation operator is proposed where the probability of the mutation decreases smoothly with the increase of the generation. In paper [7], an experiment is done to anticipate the effects between the 2-opt and the mutation operator for solving the Euclidean traveling salesman problem (TSP). In paper [8] a hybrid mutation operator called "IRGIBNNM" is proposed. The main idea of the paper is to combine a knowledge-based mutation and a random-based mutation. In paper [9] GA is developed by a sequential constructive crossover. Also, 2-opt search, a proposed local search and immigration were performed to find a better solution. In paper [10] an improved strategy is proposed to blend random crossover and dynamic mutation. The author claims that it increases the diversity of the population and optimizes mutation characters. Some modified and new crossover strategies for GA are found in [11, 12]. In paper [13], stochastic hill climbing algorithm was applied for mutation process. A comparative analysis is done on mutation operators in paper [14, 15]. In paper [16], two mutation operators named as Inverted Exchange Mutation and Inverted Displacement Mutation has been combined to improve the performance of GA and compared with four different mutation operators.

The contribution of this paper is to combine several mutation operators and take the benefits of them. There are some random-based mutations such as Swap and Insertion mutation those are faster but unable to provide a high-quality solution. On the other hand, knowledge-based mutation (heuristic) such as 2-opt mutation is very effective to find an optimal solution. But, the major drawback of 2-opt is that it is computationally expensive. The novelty of this research is to develop a triggering method that can choose any mutation operator from Swap, Insertion and 2-opt Mutation. Our idea is to apply random mutation (Swap and Insertion) at the early generation of the algorithm so that it finds some feasible solution within a short time. At the later generations, we apply knowledge-based mutation (2-opt) so that the feasible solutions could be turned into an optimal solution. 2-opt is very popular for removing cycles from a TSP tour but takes a large amount of time. That is why it is wise to invest valuable computation time only to find an optimal solution at the ending of the generation. In this work, we not only considered travel cost but also considered computation time.

The paper is organized in a few chapters. Chapter 1 is the introductory chapter that provides the theoretical background, problem definition, motivation, some related works and the novelty of this work. In Chapter 2 the methodology of this research is discussed elaborately. The experimental results are discussed in Chapter 3. Chapter 3 also provides a convergence analysis to better understand the performance of different algorithms. Chapter 4 concludes the main remarks of this research with some future indications.

2 RESEARCH METHOD

In this research, the genetic algorithm is implemented with improvement in mutation operator selection method. We proposed a Mutation Triggering Method (MTM) that can take the advantages of several prominent mutation operators such as, swap mutation, insertion mutation and 2-opt mutation.

2.1 The Structure of the Proposed Algorithm

We followed the basic structure of the genetic algorithm except for the mutation operator phase. The improved algorithm is shown in Fig. 1. The steps of the algorithms are described below:

Generate Initial Random Population. In this step, a fixed number of initial population was generated without any heuristic. As the concern of this research is to find the optimal / near-optimal solution using the mutation operators, the most commonly used, random population generation method was chosen. Some other population generation method can be found in [17-20].

Calculate Fitness of the Individuals. This step calculated the quality of each individual through a fitness function. To solve TSP, the fitness value of each chromosome is calculated by Equation (1).

$$f = \frac{1}{D} \quad (1)$$

Where, D = total travel distance of a tour = $d_1 + d_2 + \dots + d_n$ and d_n = Euclidian distance between two connected cities c_i and c_{i+1} .

Stopping Criterion This step is the stopping criterion of the algorithm. The algorithm runs for a fixed number of generations (by checking the condition, Generation < Max Generation) without considering the fact that it finds a better solution or not.

Selection of Individuals Using Tournament Selection. In this research, the Tournament Selection method is implemented since it has several benefits than other selection strategies [21]. Tournament Selection provides diversity as well as gives chance to all individuals to compete with each other.

Select Genetic Operator. In this step, crossover or mutation or both is selected according to their probability. Though in this work, the crossover operator has a fixed probability, the mutation probability p_m is adaptable with the generation. It means that the mutation probability varies with the succession of the generation.

Crossover Operator: Apply Order Crossover on Two Selected Individuals. In this part, if the crossover probability allows the operation, two selected individuals perform recombination by following the Order Crossover strategy.

Calculate Probability, p_m According to the Progression of the Generation. This step is one of the most crucial parts of the algorithm since the mutation probability p_m is calculated here. The Mutation is a very sensitive operator that cannot be changed drastically. A higher probability of mutation might end up the algorithm to divergence with large computation time. On the other hand, a very smaller mutation probability might cause local optima. That is why, the mutation probability is reduced smoothly with the succession of the generation by following the Equation (2), found in [6].

$$P_m = \frac{k}{\sqrt{\text{Current generation number}}} \quad (2)$$

Here, k is a constant factor chosen over a range of $[0, 1]$.

Mutation Selector Method. This step decides which mutation should be selected from 2-opt, Swap and Insertion mutation according to a selector equation. The 2-opt mutation is very effective to find cycles in the tour but the major drawback is that it is very slow. Applying 2-opt alone might cause greater computation time. On the other hand, Swap and Insertion mutations are very faster approach but the low-quality solutions. Therefore, in this work, these three mutation methods are combined to ensure diversity as well as lesser computation time. A selector variable is defined that decides when to choose 2-opt mutation according to the **factor** constant, shown in Equation (3). For example, if the **factor** value is set as 0.2, it means that 2-opt will be triggered on the last 20% of the generations to generate a high-quality solution, sacrificing the higher computation time. On the other hand, the beginning 80% of the generation will take the benefits (faster) of Swap and Insertion mutations. Between Swap and Insertion mutation, a fair coin toss is performed.

$$\text{selector} = \text{max. generation} - (\text{factor} * \text{max. generation}) \quad (3)$$

Mutation Operator: Apply 2-opt Mutation. This mutation operator is applied according to the decision of the Equation (3). If the generation number approaches to the end (say, last 20%) of the maximum generation, 2-opt is performed.

Mutation Operator: Apply Swap Mutation. At the beginning (say starting 80%) of the generation, Swap mutation is performed by a fair coin toss that is with 50% of probability.

Mutation Operator: Apply Insertion mutation. Insertion mutation is applied alongside the Swap mutation. It is mentioned earlier that swap mutation is applied with 50% probability. The rest of the 50% probability is applied to trigger the Insertion mutation.

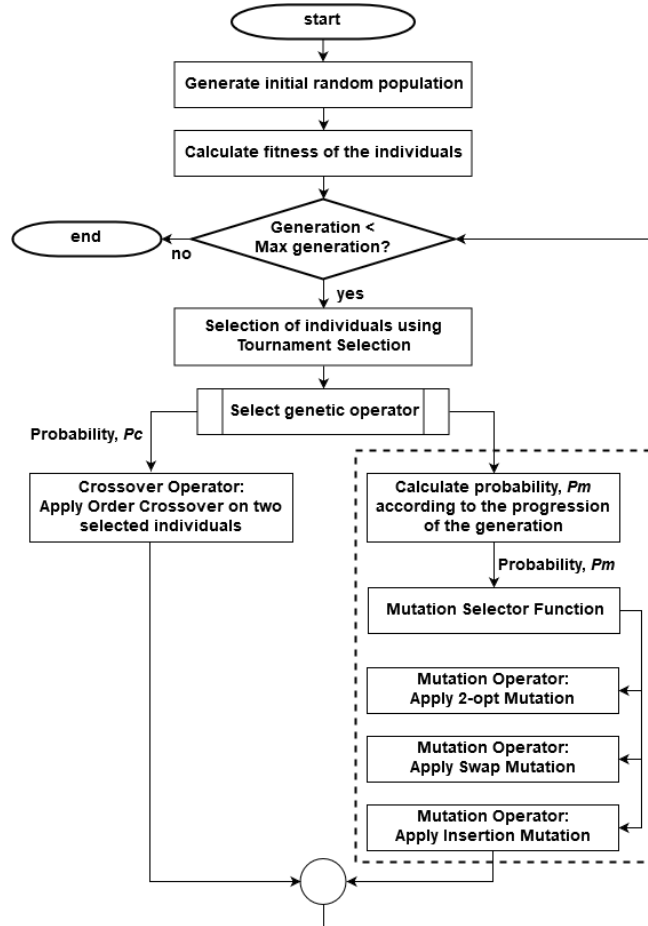


Fig. 1. The flowchart of the proposed Algorithm. The dotted line indicates the area of contribution.

2.2 The Specific Contribution of This Research: Designing the Mutation Triggering Method (MTM)

To be specific, designing MTM is the main contribution of this research. This function decides which mutation operator will be selected according to some ideas. The algorithm of MTM is shown in Algorithm 1.

Algorithm 1 MTM

```

1:  $k \leftarrow \text{range } [0, 1]$ 
2:  $\text{factor} \leftarrow \text{range } [0, 1]$ 
3: find the current generation number and maximum generation number
4: calculate the mutation probability,  $P_m = \frac{k}{\sqrt{\text{Current generation number}}}$ 
5:  $r_1 \leftarrow$  a random number,  $r_2 \leftarrow$  a random number
6: if  $r_1 < P_m$  then
7:    $\text{selector} \leftarrow \text{max generation number} - \text{factor} * \text{max generation number}$ 
8:   if  $\text{current generation number} > \text{selector}$  then
9:     Apply 2-opt mutation
10:  else if  $r_2 < 0.5$  then
11:    Apply Swap mutation
12:  else
13:    Apply Insertion mutation
14:  end if
15: end if

```

First of all, a constant number k and a multiplying factor are chosen from the range of 0 to 1. Then the current generation number and the maximum generation number is found out. Next, the p_m is calculated by dividing k by the square root of the current generation number. As the generation number increases, the denominator value becomes larger, resulting in the p_m to be smaller. It implies that with the succession of the generation the probability of mutation, p_m is decreased to ensure convergence. The constant k plays the role to distribute the value under a probabilistic range 0 to 1. Then a random number r_1 is generated (between 0 to 1) and checked whether it is less than p_m . When p_m allows the mutation operation the algorithm concentrates on which mutation to be applied. Since in this work one mutation operator is applied at a time, the total generation was divided into two parts. For example, the first part is the starting 80% of the maximum generation whereas the second part is the last 20%. If the current generation is among the last 20% of the maximum generation, the algorithm applies 2-opt mutation. This is because it is logical to spend valuable computation time on near-optimal solutions rather than random solutions. However, if the current generation lies in the first 80% of the generation, the algorithm does not concentrate on optimality rather diversity. That is why Swap mutation and Insertion mutation is applied by 50% of probability to ensure a quicker solution.

3 RESULTS AND DISCUSSIONS

This work implemented the traditional genetic algorithm (T-GA), the random crossover and dynamic mutation genetic algorithm (RCDM-GA) [10] and the proposed mutation triggering method genetic algorithm (MTM-GA) to solve nine instances from TSPLIB [22] dataset. T-GA adopted swap mutation, insertion mutation and 2-opt mutation separately. RCDM-GA also adopted swap mutation and insertion mutation. Due to very high computational time, the 2-opt mutation was not used for RCDM-GA. Finally, MTM-GA was implemented with the combined mutations. Both travel cost and computation time were considered while these algorithms experimented.

3.1 Experimental Result Based on Travel Cost

The optimal solutions considering travel cost is represented in Table 1. From Table 1 it is seen that T-GA adopting 2-opt mutation found best solutions 2727, 43702 and 2419 for a280, lin318 and rat195 dataset instances respectively. But, most of the optimal solution was found by MTM-GA. It was successful to find the optimal solution for eil51 (432), bier127 (119416), kroa100 (21666), kroa200 (30697), pr226 (80728) and st70 (678) instances. However, T-GA (Swap), T-GA (Insertion), RCDM-GA (Swap) and RCDM-GA (Insertion) ended up in local optima.

Table 1. Optimal solution (travel cost) for different algorithms and mutation operators

Instances	Optimal	T-GA			RCDM-GA		MTM-GA
		Swap	Insertion	2-Opt	Swap	Insertion	Combined
eil51	426	754	871	433	551	621	432
a280	2579	20753	21476	2727	151518	27779	2756
bier127	118282	365293	374119	123294	469925	411091	119416
kroa100	21282	82631	79085	21846	149170	69010	21666
kroa200	29368	199256	190549	30971	234087	242155	30697
lin318	42029	384807	388311	43702	1479572	698061	44563
pr226	80369	1025700	976004	81132	1228103	1089287	80728
st70	675	1886	1680	681	1133	31093	678
rat195	2323	13061	13321	2419	19665	87280	2450

3.2 Experimental Result Based on Computation Time

In this paper, we considered computation time because it is also important to find out which algorithm finds the optimal solution in a considerable amount of time. Since Swap and Insertion mutations are the random-based approach, the algorithms adopting those mutation operators are the fastest. From Table 2 it is clear that all the minimum computation time was taken by either the T-GA (Swap) or the T-GA (Insertion).

RCDM-GA (Swap) and RCDM-GA (Insertion) also takes very less computation time. However, the computationally most expensive approach is the T-GA (2-opt) algorithm. MTM-GA takes a considerable amount of time.

Table 2. Computation time for different algorithms and mutation operators

Instances	T-GA			RCDM-GA		MTM-GA
	Swap	Insertion	2-Opt	Swap	Insertion	Combined
eil51	2.3	2.48	13.23	4.09	5.94	5.3
a280	40.11	42.06	1313.95	49.09	64.69	354.79
bier127	10.67	10.63	157.77	13.68	19.65	50.54
kroa100	6.38	6.85	93.92	9.58	13.29	27.16
kroa200	21.35	22.92	657.78	27.28	29.63	185.74
lin318	51.07	52.46	2453.97	53.13	81.28	663.22
pr226	26.22	27.02	992.72	33.77	43.85	229.39
st70	4.06	3.88	31.67	6.39	9.56	11.55
rat195	21.44	21.46	570.57	25.43	36.15	155.37

3.3 Convergence Analysis

We considered one instance rat195 to experiment how all of the algorithms converge with the progression of the generation. After 1000 generation the solution for each generation was shown in Fig. 2. It can be concluded that because of being random in nature, T-GA (Swap) and T-GA (Insertion) quickly get stuck in local optima, implies that unable to bring diversity. Since T-GA (2-opt) is a heuristic approach, it finds the optimal/ near-optimal solution within the first few generations. Running the algorithm for a long generation does not bring much diversity in this case. On the other hand, RCDM- GA (Swap) and RCDM-GA (Insertion) is too much random in nature. Since the mutation probability calculating function of RCDM-GA increases the probability when the solution does not vary, it fluctuates the solution for large dataset instances. It is generally effective for small dataset instances. However, MTM-GA is designed in such a way that it can use the benefits of both randomness and knowledge-based mutation operations. It quickly converges to the local optima using the random mutations. To escape from local optima, it uses heuristic mutation such as 2-opt mutation. The major improvement of TMT-GA is that it concentrates not only on optimality but also in computation time.

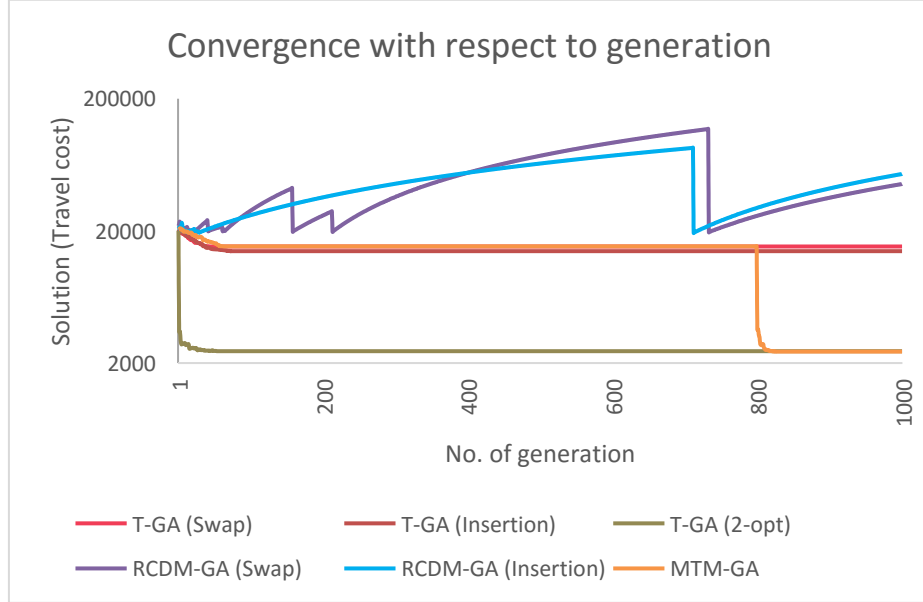


Fig. 2. The convergence of T-GA (Swap), T-GA (Insertion), T-GA (2-opt), RCDM-GA (Swap), RCDM-GA (Insertion) and MTM-GA algorithms after 1000 generation for the rat195 instance.

4 Conclusion and Future Work

Genetic algorithm is a very flexible algorithm that allows independence in operator design. Using random mutation cannot bring enough diversity when the algorithm needs it most. On the other hand, unnecessary mutation (especially when the algorithm is about to converge) might cause the wrong solution. Being too much heuristic-based again costs too much of computation time even though it is good for optimality. Our approach was to trigger the random and heuristic approach by understanding the generation of the algorithm. The experimental result proved that our proposed method can find an optimal solution. The T-GA (2-opt) is also able to find an optimal solution like the proposed method does. But, the novelty of the proposed algorithm is that it can find the solution in a very less amount of time whereas other approaches are computationally expensive.

In future, we would like to implement more varieties of mutation operators to check the effect of the genetic algorithm. Implementing more measures to detect the rate of solution quality and finding the local optima might be a good direction to design the triggering method. It will be a great future research indication to make the triggering method recommending the appropriate mutation operator to escape from the local optima based on experience.

References

1. Matai, R., Singh, S. P., Mittal, M. L.: Traveling salesman problem: an overview of applications, formulations, and solution approaches. Traveling salesman problem, theory and applications, InTech, Croatia (2010).
2. Shukla, A., Pandey, H. M., Mehrotra, D.: Comparative review of selection techniques in genetic algorithm. In: 2015 International Conference on Futuristic Trends on Computational Analysis and Knowledge Management (ABLAZE), pp. 515-519 (2015).
3. Larranaga, P., Kuijpers, C. M., Murga, R. H., Inza, I., Dizdarevic, S.: Genetic algorithms for the travelling salesman problem: A review of representations and operators. Artificial Intelligence Review 13(2), 129-70 (1999).
4. Sathya, N., Muthukumaravel, A.: A Review of the Optimization Algorithms on Traveling Salesman Problem. Indian Journal of Science and Technology 8(1), (2015).
5. Tollis, I. G.: Chapter 10 The Traveling Salesman Problem, <https://www.csd.uoc.gr/~hy583/papers/ch11.pdf>, last accessed 2019/03/10.
6. Maity, S., Roy, A., Maiti, M.: A rough multi-objective genetic algorithm for uncertain constrained multi-objective solid travelling salesman problem. Granular Computing 4(1), 125- 42 (2019).
7. Chang, C.: A 2-opt with mutation operator to the traveling salesman problem. Int J Adv Eng Technol Comput Sci (IROSSS) 2(1), 16-21 (2015).
8. Alkafaween, E., Hassanat, A.: Improving TSP Solutions Using GA with a New Hybrid Mutation Based on Knowledge and Randomness. arXiv preprint arXiv:1801.07233 (2018).
9. Ahmed, Z. H.: A hybrid genetic algorithm for the bottleneck traveling salesman problem. ACM Transactions on Embedded Computing Systems (TECS) 12(1), 9 (2013).
10. Xu, J., Pei, L., Zhu, R. Z.: Application of a genetic algorithm with random crossover and dynamic mutation on the travelling salesman problem. Procedia computer science 131, 937-945 (2018).
11. Singh, D. R., Singh, M. K., Singh, T., Prasad, R.: Genetic algorithm for solving multiple traveling salesmen problem using a new crossover and population generation. Computación y Sistemas 22(2), (2018).
12. Hussain, A., Muhammad, Y. S., Nauman Sajid, M., Hussain, I., Mohamd Shoukry, A., Gani, S.: Genetic algorithm for traveling salesman problem with modified cycle crossover operator. Computational intelligence and neuroscience 2017, (2017).
13. Katayama, K., Sakamoto, H., Narihisa, H.: The efficiency of hybrid mutation genetic algorithm for the travelling salesman problem. Mathematical and Computer Modelling 31(10-12), 197-203 (2000).
14. Abdoun, O., Abouchabaka, J., Tajani, C.: Analyzing the performance of mutation operators to solve the travelling salesman problem. arXiv preprint arXiv:1203.3099 (2012).
15. Albayrak, M., Allahverdi, N.: Development a new mutation operator to solve the traveling salesman problem by aid of genetic algorithms. Expert Systems with Applications 38(3), 1313-1320 (2011).
16. Deep, K., Mebrahtu, H.: Combined mutation operators of genetic algorithm for the travelling salesman problem. International Journal of Combinatorial Optimization Problems and Informatics 2(3), 1-23 (2011).
17. Toğan, V., Daloğlu, A. T.: An improved genetic algorithm with initial population strategy and self-adaptive member grouping. Computers & Structures 86(11-12), 1204-1218 (2008).
18. Maaranen, H., Miettinen, K., Mäkelä, M. M.: Quasi-random initial population for genetic algorithms. Computers & Mathematics with Applications 47(12), 1885-1895 (2004).
19. Diaz-Gomez, P. A., Hougen, D. F.: Initial Population for Genetic Algorithms: A Metric Approach. In Gem, 43-49 (2007).
20. Deng, Y., Liu, Y., Zhou, D.: An improved genetic algorithm with initial population strategy for symmetric TSP. Mathematical Problems in Engineering, (2015).
21. Saini, N.: Review of selection methods in genetic algorithms. International Journal Of Engineering And Computer Science 6(12), 22261-22263 (2017).
22. Reinelt, G.: TSPLIB—A traveling salesman problem library. ORSA journal on computing 3(4), 376-84 (1991).